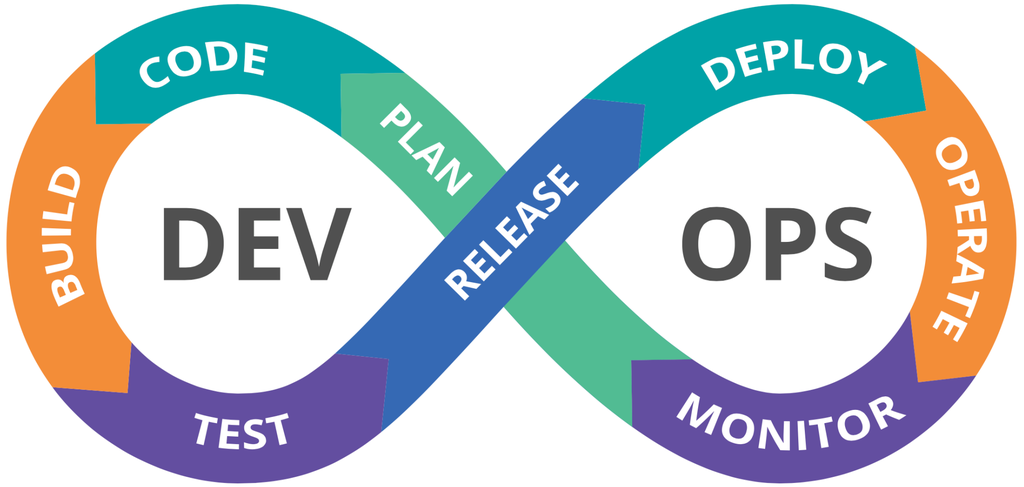


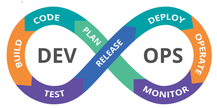
Bezbednost kao svakodnevni posao svih zaposlenih

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



Informaciona bezbednost kao deo DevOps principa

- DevOps principi mogu da ponude jako dobre odgovore na to kako integrisati informacionu bezbednost u svakodnevne radne zadatke.
  - često se informaciona bezbednost navodi kao razlog za neusvajanje DevOps principa razvoja
  - kada se poslovi informacione bezbednosti organizuju tako da predstavljaju izolovanu organizacionu jedinicu, izvan svakodnevnog lanca isporuke softvera to dovodi do raznoraznih ograničenja i problema



Informaciona bezbednost kao deo DevOps principa

- Integracija informacione bezbednosti u svakodnevne poslove razvoja softvera podrazumeva
  - integraciju bezbednosnih aspekata u razvojni proces i integrisanje njihovog prikaza u demonstraciju svake razvojne iteracije
  - integraciju bezbednosnih pitanja u praćenje i analizu defekata
  - integraciju preventivne bezbednosne kontrole na repozitorijume izvornog koda i kontrolu deljenih biblioteka
  - integrisanje bezbednosnih aspekata u *deployment pipeline*
  - obezbeđivanje same aplikacije
  - obezbeđivanje kompletnog lanca nabavke softvera
  - obezbeđivanje izvršnih okruženja
  - integraciju bezbednosnih aspekata u telemetriju
    - kreiranje telemetrije za bezbednosne probleme u aplikaciji
    - krerianje telemetrije za bezbednosne probleme izvršnih okruženja
  - zaštitu samog *deployment pipeline-a*



Integracija bezbednosnih aspekata u razvojni proces i njihov prikaz u demonstraciji svake razvojne iteracije

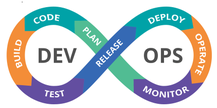
- Jedan od primarnih ciljeva jeste da se pitanja informacione bezbednosti integrišu u razvojni ciklus što ranije, umesto da se informaciona bezbednost “dodaje” na kraju razvojne faze

- Jedan od načina je da se članovi timova zaduženih za informacionu bezbednost uvek pozivaju na demo sastanke na kraju svake razvojne faze

  - Na ovaj način oni su dublje uključeni i bolje razumeju šta su ciljevi razvoja u određenim fazama, mogu da uoče potencijalne probleme i obezbede sugestije kako ugraditi odgovarajuće bezbednosne mehanizme u tekućoj fazi razvoja.

    - Ovo je mnogo efikasniji princip, nego da se bezbednosni aspekti razmatraju tek pred isporuku, čime se može desiti da iz bezbednosnih razloga članovi InfoSec tima stave blokadu na samu isporuku, dok se određeni problemi ne reše.

    - Kada su članovi InfoSec timova od starta uključeni u razvoj bilo kog novog modula, moguće je da se bezbednosni problemi na vreme uoče i odgovarajuće zaštitne mere primene još tokom razvoja softvera.



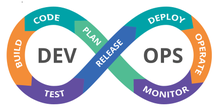
Integracija bezbednosnih pitanja u praćenje i analizu defekata

- Kad god je to moguće, potrebno je obezbediti da se bezbednosni problemi prate kroz isti sistem koji koriste i razvojni i operativni timovi. Na taj način svima je vidljivo koje bezbednosne probleme treba rešavati i moguće je postaviti prioritete za rešavanje uzimajući u obzir i ove probleme.

  - ovo je odmak od klasičnog principa gde su se bezbednosni problemi evidentirali u zasebnim GRC (governance, risk, and compliance) alatima i bili praktično nevidljivi članovima razvojnih i operativnih timova.

  - sada se bezbednosni problemi upisuju u radne zadatke kao i svi ostali (npr. u Jira-i), i vrlo često im se dodeljuje visok prioritet kako bi se rešavali što brže.

  - osim toga kada god se desi neki bezbednosni problem radi se i post mortem analiza, kako bi se kao i u slučaju ostalih grešaka, utvrdili načine kako izbeći njihovo ponavljanje u budućnosti

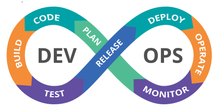


Integracija preventivne bezbednosne kontrole na repozitorijume izvornog koda i kontrolu deljenih biblioteka

- Na repozitorijume izvornog koda osim samog koda aplikacije, alata za development i deployment, konfiguracija, neophodno je dodati i sve što je povezano sa bezbednosnim konceptima - bezbednosne mehanizme koji se implementiraju, odobrene (od strane InfoSec timova) deljene biblioteke, kao i alate koji služe za obezbeđivanje kako same aplikacije koja se razvija tako i kompletnih razvojnih okruženja.

  - Kreiranje deljene “bezbednosne platforme” - standardnih komponenti koje se koriste za autentikaciju, autorizaciju, upravljanje lozinkama, enkripciju podataka, logovanje i druge bezbednosne servise koji se mogu koristiti na raznim mestima. Na ovaj način se osigurava da različiti softverski proizvodi sigurno imaju implementirano konzistentne servise koji su provereni i odobreni.

  - Osim toga neophodno je obezbediti redovne obuke u poglednu bezbednosnih aspekata za sve članove razvojnih i operativnih timova.



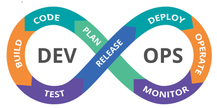
Integracija preventivne bezbednosne kontrole na repozitorijume izvornog koda i kontrolu deljenih biblioteka

- Na sistemima za kontrolu verzija (repozitorijume koda) treba dodati:
  - odobrene biblioteke koje se koriste za određene bezbednosne aspekte (npr. 2FA, MFA)
  - sve što je u vezi sa upravljanjem “tajnim” podacima (ključevima, podešavanjima za sigurne konekcije i sl.)
  - OS pakete i odobrene nadogradnje, posebno sve osetljive bezbednosne zakrpe
- Praktično sve što se koristi za “hardening” naših softverskih rešenja



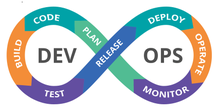
Integrisanje bezbednosnih aspekata u deployment pipeline

- Klasični pristup je bio da se bezbednosni aspekti nekog softverskog rešenja analiziraju na kraju razvojne faze.
  - Često rezultujući opširnim dokumentima sa analizom nađenih nedostataka.
  - Ovi zaključci često ne bi bili odmah adresirani (zbog fiksnih termina isporuke verzija), ili prosto zato što su propusti otkriveni prekasno da bi se mogli rešiti relativno lako i u kratkom vremenu.
- DevOps pristup je da se ova pitanja sagledaju što ranije, da se bezbednosni problemi uoče i rešavaju tokom same razvojne faze.



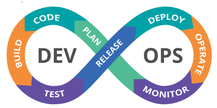
Integrisanje bezbednosnih aspekata u deployment pipeline

- Jedan od ključnih alata za postizanje ovog cilja je da se u skupove automatizovanih testova obavezno dodaju testovi koji testiraju bezbednosne aspekte aplikacije
  - idealno ovi testovi se, kao i svi ostali, izvršavaju automatski nakon svakog commit-a u glavnu razvojnu granu
  - alati kao što su GauntIt omogućavaju potpunu integraciju u deployment pipeline
- Na ovaj način vrlo lako je uočiti koji testovi su “pali” i korigovati uzroke u samom začetku.



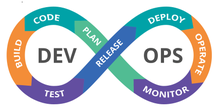
Obezbeđivanje same aplikacije

- Razvojni timovi se suštinski fokusiraju na ispunjavanje funkcionalnih zahteva, vrlo često analizirajući samo “pozitivne putanje izvršavanja” - *happy flow.*
  - QA, InfoSec se koncentrišu i na testiranje izvršavanja u slučajevima kada postoje greške, posebno u slučajevima kada su te greške bezbednosne prirode (namerni pokušaji korišćenja neodgovarajućih kredencijala, pristup resursima za koje nismo autorizovani i sl.) - *sad paths.*
  - Idealno svi testovi ove prirode bi trebalo da su automatizovani



Obezbeđivanje same aplikacije

- Testovi treba da obuhvataju:
  - statičku analizu koda - testovi se analiziraju bez izvršavanja - analizira se kod sa stanovišta prepoznavanja upotrebe pogrešnih praksi, loših i nesigurnih algoritama, potencijalno malicioznog koda ili *backdoor* delova koda.
  - dinamička analiza - analiza ponašanja aplikacije tokom izvršavanja - prati se npr. zauzeće memorije, funkcionalnost, performanse. Na ovaj način može se sagledati način na koji bi i maliciozni korisnik mogao sagledati aplikaciju i uočiti slabe tačke.
  - analiza zavisnosti (*dependancy scanning*) - još jedan tip statičke analize, gde se utvrđuju sve biblioteke koje se koriste, te proverava da li se koriste samo one koje su prethodno odobrene
  - provera integriteta izvornog koda i potpisivanje koda - svi članovi razvojnog tima bi trebali da imaju svoj ključ kojim potpisuju kod, i svi komitovi bi trebali da budu digitalno potpisani. Osim toga svi isporučivi artifakti (instalacioni paketi) koje proizvede CI proces, trebali bi biti potpisani i njihov hash evidentiran.
- Implemetirati dobre prakse za čuvanje lozinki, logiku obrade zaboravljenih lozinki, upravljanje procesima logovanja, uklanjanje XSS ranjivosti i sl.



Obezbeđivanje kompletnog lanca nabavke softvera

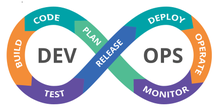
- Razvojni timovi danas često ne pišu potpuno novi proizvod prilagođen potrebama samo jednog korisnika, već se proizvodnja novog softvera često oslanja na “sklapanje” novog rešenja iz postojećih komponenti
  - drugim rečima oslanjamo se na postojeće softvere i biblioteke, nasleđujući od njih ne samo njihovu funkcionlanost već i sve njihove bezbednosne ranjivosti

- Stoga je neophodno obratiti pažnju na izbor biblioteka koje se koriste, kao i na edukaciju razvojnih timova da strane biblioteke koriste sa odgovarajućim stepenom opreza, te da biblioteke biraju tako da se oslanjaju na one za koje postoji istorija da su uočeni i prijavljeni problemi brzo prijavljivani i korektno rešavani (što je često slučaj sa OpenSource softverom).
- Takođe neophodno je analizirati upotrebu različitih verzija istih bilioteka i težiti ka tome da svi projekti koriste najnovije verzije u kojima su potencijalni problemi uočeni i uklonjeni.



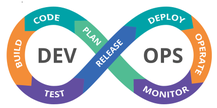
Obezbeđivanje razvojnih i izvršnih okruženja

- Neophodno je učiniti sve što je u našoj moći da su izvršna okruženja obezbeđena i sa minimalnim stepenima rizika. Iako smo kreirali dobre i potvrđene konfiguracije, neophodno je imati odgovarajući monitoring koji će pratiti da su aktivna okruženja i dalje u saglasnosti sa željenim i postavljenim konfiguracijama, koje garantuju određeni nivo bezbednosti.
- Ovo možemo rešiti tako što kreiramo automatizovane testove koji proveravaju sve relevantne postavke u konfiguracijama aktivnih izvršnih okruženja.
- Takođe testove koristimo da skeniramo naša okruženja na poznate ranjivosti.
  - Jako je bitno biti potpuno svestan stvarnog stanja izvršnih okruženja - utvrditi otvorene portove, utvrditi da li su primenjena osiguranja protiv npr. SQLInjection napada. Koriste se razni alati koji mogu da daju izveštaje o stvarnom stanju konfiguracije.



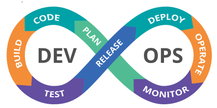
Integracija bezbednosnih aspekata u telemetriju

- Često protekne dosta vremena od momenta kada se bezbednosni incident desi do momenta kada se otkrije da se on desio. Ova činjenica ukazuje na to da je moguć dosta dug vremenski period u kome se određeni propusti mogu maliciozno iskoristiti.
- Da stvar bude gora, često samo otkrivanje bezbednosnog propusta nije rezultat rigoroznog internog monitoringa, već se prijavi izvana, najčešće od strane poslovnog partnera koji je primetio “čudne transakcije”.
  - Jedan od razloga za ovo je što niko u organizaciji ne pregleda redovno log fajlove
  - Interna kontrola često nije dovoljno efikasna da otkrije bezbednosne propuste
- Kao što su odgovarajući telemetrijski podaci od esencijalne važnosti za razumevanje i uklanjanje problema u funkcionisanju aplikacije tako je neophodno obezbediti odgovarajuće telemetrijske podatke koji ukazuju na pojavu bezbednosnih problema u aplikaciji



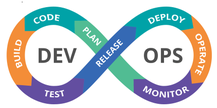
Kreiranje telemetrije za bezbednosne probleme u aplikaciji

- Podaci koji mogu da otkriju problematično ponašanje korisnika aplikacije
  - broj uspešnih i neuspešnih pokušaja prijavljivanja na aplikaciju
  - resetovanje lozinki
  - resetovanje korisničkog emaila
  - promene sačuvanih kritičnih (npr. o kreditnim karticama) i ličnih podataka



Kreiranje telemetrije za bezbednosne probleme izvršnih okruženja

- Nije dovoljno instrumentalizovati samo našu aplikaciju kako bi povećali bezbednost. Jednako važno je imati pravovremene informacije koje ukazuju na stanje izvršnih okruženja.
- Monitoring od interesa:
  - izmene OS-a
  - promene u bezbednosnim grupama (Security groups)
  - izmene konfiguracija (npr. OSSEC, Puppet, Chef, Tripwire)
  - izmene u korišćenoj cloud infrastrukturi (VPC, security groups, users and privileges)
  - XSS pokušaji (“cross-site scripting attacks”)
  - SQLi pokušaji (“SQL injection attacks”)
  - greške sa Web servera(4XX i 5XX errors)”



Zaštita samog deployment pipeline-a

- Infrastruktura za naš CI i CD pipeline predstavlja novu “metu” za napade.
  - neko može da kompromituje server koji upravlja *deployment pipeline-om* i iskoristi kredencijale za pristup sistemu za verzioniranje koda - onda može da preuzme sav kod, ili može namerno da ga uništi, ili da u kod doda maliciozne izmene koje bi se isporučile našem krajnjem klijentu - trajna šteta.
  - jedno od mesta gde se može sakriti maliciozni kod su npr. Unit testovi - retko ko ih ponovo analizira, a izvršavaju se pri svakom ciklusu *deployment pipelina*



Zaštita samog deployment pipeline-a

- Strategije za zaštitu mogu da obuhvate:
  - Hardening CB, CI, i CD servera, ali i obezbeđivanje mogućnosti da se ovi serveri brzo “podignu” u istovetnoj konfiguraciji ako to ustreba (slike virtuelnih mašina sa preinstaliranim i konfigurisanim potrebnim alatima).
  - Revizija svih izmena koje se unose u sistem za verzioniranja. Na ovaj način se obezbeđuje da na CI servere ne dospe maliciozni kod.
  - Instumentalizacija repozitorijuma da može da prepozna ukoliko neki kod sadrži sumnjive pozive API-ja (unit testovi prisupaju fajlvocima ili izvršavaju mrežne pozive).
  - Svaki CI proces treba da je konfigurisan tako da se izvršava u izolovanom kontejeneru ili na izolovnaoj VM.
  - Osigurati da su kredencijali kojima obični koristi pritupaju CI sistemu budu samo za čitanje (read-only).
- Rekategorisanje svih low-risk izmena u standardne - faktički posmatrati svaki izmenu u pipelinu kao potencijalno problematičnu.
- Obezbediti dokumentaciju za auditore, i dokumentaciju koja potvrđuje poštovanje standarda